



```
!pip install sentence-transformers textblob
```

```
Requirement already satisfied: sentence-transformers in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: textblob in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: transformers<6.0.0,>=4.41.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: huggingface-hub>=0.20.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: torch>=1.11.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: typing_extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: nltk>=3.9 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: shellingham in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: typer>=0.24.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages
```

```
import random
import time
import re
from textblob import TextBlob
```

```

from sentence_transformers import SentenceTransformer, util

def detect_emotion(text):
    sentiment = TextBlob(text).sentiment.polarity

    if sentiment > 0.3:
        return "positive"
    elif sentiment < -0.3:
        return "negative"
    else:
        return "neutral"

intents = {
    "greeting": [
        "hi", "hello", "good morning", "hey there"
    ],
    "order_issue": [
        "where is my order", "order not delivered", "track my parcel"
    ],
    "refund_issue": [
        "refund", "return my money", "cancel my order"
    ],
    "technical_problem": [
        "app crashed", "error message", "not working"
    ]
}

responses = {
    "greeting": {
        "neutral": "Hi there! How can I help you today?",
        "positive": "Hey! 😊 Great to see you. What can I do for you?",
        "negative": "Hello. I hope everything is okay. Tell me how I can assist",
    },

    "order_issue": {
        "neutral": "Let me quickly check that for you. Could you share your order",
        "negative": "I'm really sorry for the inconvenience. Let me track it for",
        "positive": "Sure! Let's track that order together. Please share your",
    },

    "refund_issue": {
        "neutral": "Refunds are processed within 5–7 working days.",
        "negative": "I understand how frustrating that can be. I'll help you with",
        "positive": "No worries! I'll guide you through the refund process."
    },

    "technical_problem": {
        "neutral": "Can you describe the issue in more detail?",
        "negative": "I'm really sorry you're facing this issue. Let's fix it together."
    }
}

```

```

        "positive": "Got it! Let's solve this quickly. What exactly is happen
    },
    "fallback": {
        "neutral": "I want to make sure I give you accurate information. Let
        "negative": "I don't want to give you incorrect guidance. I'll immedi
        "positive": "Let me double-check this with our support team to ensure
    }
}

```

```
model = SentenceTransformer('all-MiniLM-L6-v2')
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning: The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab. You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models.
warnings.warn(

```

```
modules.json: 0%|          | 0.00/349 [00:00<?, ?B/s]
```

```
config_sentence_transformers.json: 0%|          | 0.00/116 [00:00<?, ?B/s]
```

```
README.md: 0.00B [00:00, ?B/s]
```

```
sentence_bert_config.json: 0%|          | 0.00/53.0 [00:00<?, ?B/s]
```

```
config.json: 0%|          | 0.00/612 [00:00<?, ?B/s]
```

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a token in your environment.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a token in your environment.

```

```
model.safetensors: 0%|          | 0.00/90.9M [00:00<?, ?B/s]
```

Loading weights: 0%| | 0/103 [00:00<?, ?it/s]

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2

| Key                     | Status     |  |  |
|-------------------------|------------|--|--|
| -----+-----+-----+----- |            |  |  |
| embeddings.position_ids | UNEXPECTED |  |  |

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture

tokenizer\_config.json: 0%| | 0.00/350 [00:00<?, ?B/s]

vocab.txt: 0.00B [00:00, ?B/s]

tokenizer.json: 0.00B [00:00, ?B/s]

special\_tokens\_map.json: 0%| | 0.00/112 [00:00<?, ?B/s]

config.json: 0%| | 0.00/190 [00:00<?, ?B/s]

```
def detect_intent(user_input):
    user_embedding = model.encode(user_input, convert_to_tensor=True)

    best_intent = None
    highest_score = 0

    for intent, examples in intents.items():
        example_embeddings = model.encode(examples, convert_to_tensor=True)
        similarity = util.cos_sim(user_embedding, example_embeddings)
        score = float(similarity.max())

        if score > highest_score:
            highest_score = score
            best_intent = intent

    return best_intent, highest_score

def detect_intent(user_input):
```

```

user_embedding = model.encode(user_input, convert_to_tensor=True)

best_intent = None
highest_score = 0

for intent, examples in intents.items():
    example_embeddings = model.encode(examples, convert_to_tensor=True)
    similarity = util.cos_sim(user_embedding, example_embeddings)
    score = float(similarity.max())

    if score > highest_score:
        highest_score = score
        best_intent = intent

return best_intent, highest_score

conversation_history = []

def generate_response(user_input):
    emotion = detect_emotion(user_input)
    intent, confidence = detect_intent(user_input)

    conversation_history.append(user_input)

    if confidence < 0.45:
        intent = "fallback"

    reply = responses[intent][emotion]

    return reply, confidence, emotion

def human_typing(text):
    for char in text:
        print(char, end="", flush=True)
        time.sleep(0.02)
    print()

print("Empathetic Cognitive Support Agent Activated.")
print("Type 'exit' to end conversation.\n")

while True:
    user_input = input("You: ")

    if user_input.lower() == "exit":
        print("Session ended.")
        break

    reply, confidence, emotion = generate_response(user_input)

```

```
print("Bot: ", end="")
human_typing(reply)
```

```
print(f"(Emotion Detected: {emotion} | Confidence: {round(confidence,2)})")
print("-"*50)
```

Empathetic Cognitive Support Agent Activated.  
Type 'exit' to end conversation.

You: exit  
Session ended.